

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: pf	Course Code: CMPE-112L
Assignment Type: lab	Dated: 01-06-2026
Semester: 1	Session: Spring 2026
Lab/Project/Assignment #: 14	CLOs to be covered: CLO 2
Lab Title: os libraries, if_name_=="_main_", is vs==	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

Lab Goals/Objectives:

- To understand the use of Python's `os` library for interacting with the operating system.
- To learn the purpose and implementation of the `if __name__ == "__main__"` statement.
- To understand the use of the `is` operator and how it differs from the `==` operator.
- To develop practical skills by implementing these concepts in Python programs.

Equipment Required:

- Laptop/PC
- Python 3.x
- VS Code (or any Python IDE)
- Windows Operating System

Theory:

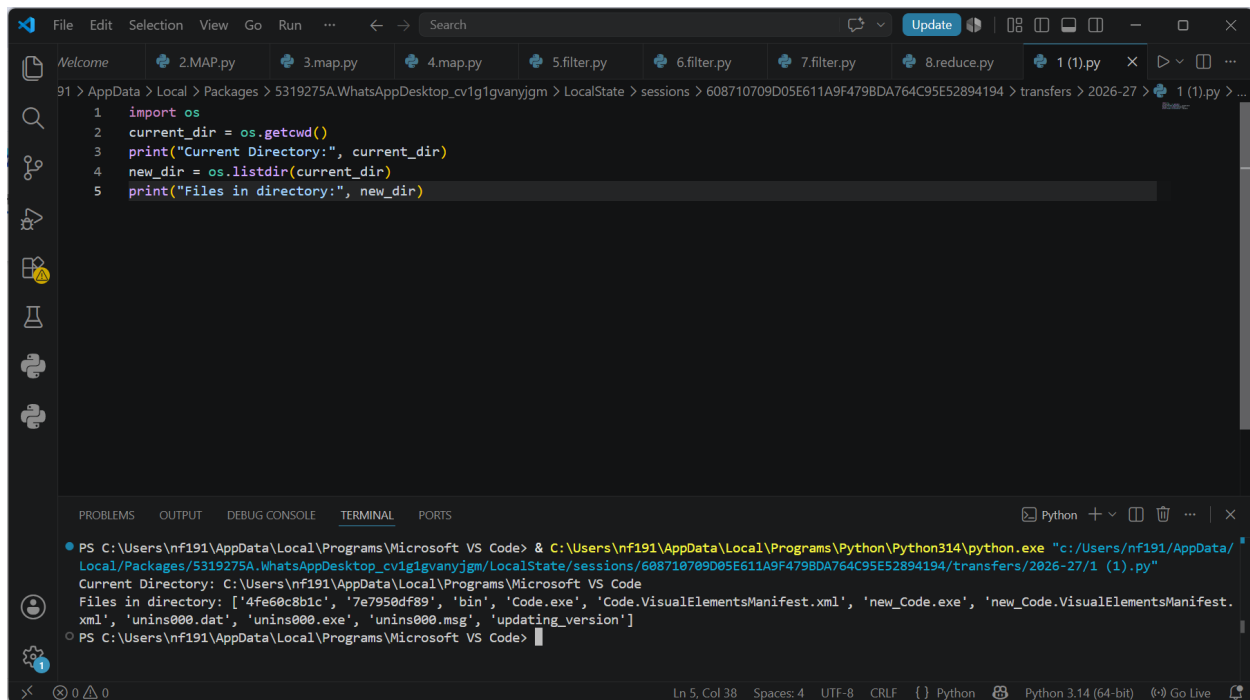
The `os` module in python provides functions to interact with the operating system, such as retrieving the current working directory, listing files and folders, creating and removing directories, and checking whether files exist. The `if __name__ == "__main__"` statement ensures that specific code is executed only when a python file is run directly, not when it is imported as a module. Python also provides the `==` operator to compare the values of objects and the `is` operator to compare whether two variables refer to the same object in memory. Understanding these concepts helps in writing organized, efficient, and reusable Python programs.

Lab Work:

Task 1:

Write a program to use the `os` library to display the current working directory and list all files and folders present in that directory.

Result:



The image shows a Visual Studio Code editor window with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, and a search bar. The Explorer sidebar on the left shows a file tree with folders like 'AppData' and 'Local', and files like '2.MAP.py', '3.map.py', '4.map.py', '5.filter.py', '6.filter.py', '7.filter.py', '8.reduce.py', and '1 (1).py'. The main editor area displays a Python script with the following code:

```
1 import os
2 current_dir = os.getcwd()
3 print("Current Directory:", current_dir)
4 new_dir = os.listdir(current_dir)
5 print("Files in directory:", new_dir)
```

The bottom panel shows the TERMINAL output, which includes the command to run the script and its output:

```
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/AppData/Local/Packages/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F4798DA764C95E52894194/transfers/2026-27/1 (1).py"
Current Directory: C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code
Files in directory: ['4fe60c8b1c', '7e7950df89', 'bin', 'Code.exe', 'Code.VisualElementsManifest.xml', 'new_Code.exe', 'new_Code.VisualElementsManifest.xml', 'unins000.dat', 'unins000.exe', 'unins000.msg', 'updating_version']
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code>
```

The status bar at the bottom indicates the current file is 'Ln 5, Col 38', the encoding is 'UTF-8', and the language is 'Python'.

Task 2:

Write a program to use the `os` library to create a new directory, display a confirmation message, and then remove the directory.

Result:

The image shows a Windows IDE interface with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, and a search bar. Below the menu is a toolbar with icons for file operations and a search icon. The main editor area displays a Python script with the following code:

```
s > nf191 > AppData > Local > Packages > 5319275A.WhatsAppDesktop_cv1g1gvanyjgm > LocalState > sessions > 608710709D05E611A9F479BDA764C95E52894194 > transfers > 2026-27 > 2.py
1 import os
2 folder = "test_folder"
3 os.mkdir(folder) # create folder
4 print(f"Folder '{folder}' created")
5 os.rmdir(folder) # remove folder
6 print(f"Folder '{folder}' removed")
```

Below the editor is a panel with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active, showing the execution of the script:

```
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nf191/AppData/Local/Packages/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F479BDA764C95E52894194/transfers/2026-27/2.py
Folder 'test_folder' created
Folder 'test_folder' removed
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> |
```

The status bar at the bottom shows the current cursor position (Ln 6, Col 36), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), the active language (Python), the Python version (Python 3.14 (64-bit)), and the Go Live status.

Task 3:

Write a program to use the `os` library to check whether a file exists and display the name of the operating system.

Result:

The screenshot displays the Visual Studio Code interface. The main editor window contains a Python file named `3.py`. The code defines a variable `path = "data.txt"` and uses `os.path.exists(path)` to check for its presence. Depending on the result, it prints either "File exists" or "File does not exist". Below the code, the Output window shows the execution results from a terminal session, indicating that the file `data.txt` does not exist.

```

import os
path = "data.txt"
if os.path.exists(path):
    print("File exists")
else:
    print("File does not exist")
print("OS name:", os.name)

```

Output:

```

PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nf191/AppData/L...ocal/Packages/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D0E611A9F479BDA764C95E52894194/transfers/2026-27/3.py
File does not exist
OS name: nt

```

Task 4:

Write a program to demonstrate the use of the `if __name__ == "__main__"` statement by executing a function only when the Python file is run directly.

Result:

[illegible]

Task 5:

Write a program to demonstrate the use of the `if __name__ == "__main__"` statement by calling a function to perform addition and display the result only when the file is executed directly.

Result:

The screenshot displays the Visual Studio Code interface. The editor window shows a file named `5.filter.py` with the following Python code:

```

1  def add(a, b):
2      return a + b
3
4  if __name__ == "__main__":
5      x = 5
6      y = 20
7      result = add(x, y)
8      print("Result:", result)

```

The terminal at the bottom shows the command to run the script and its output:

```

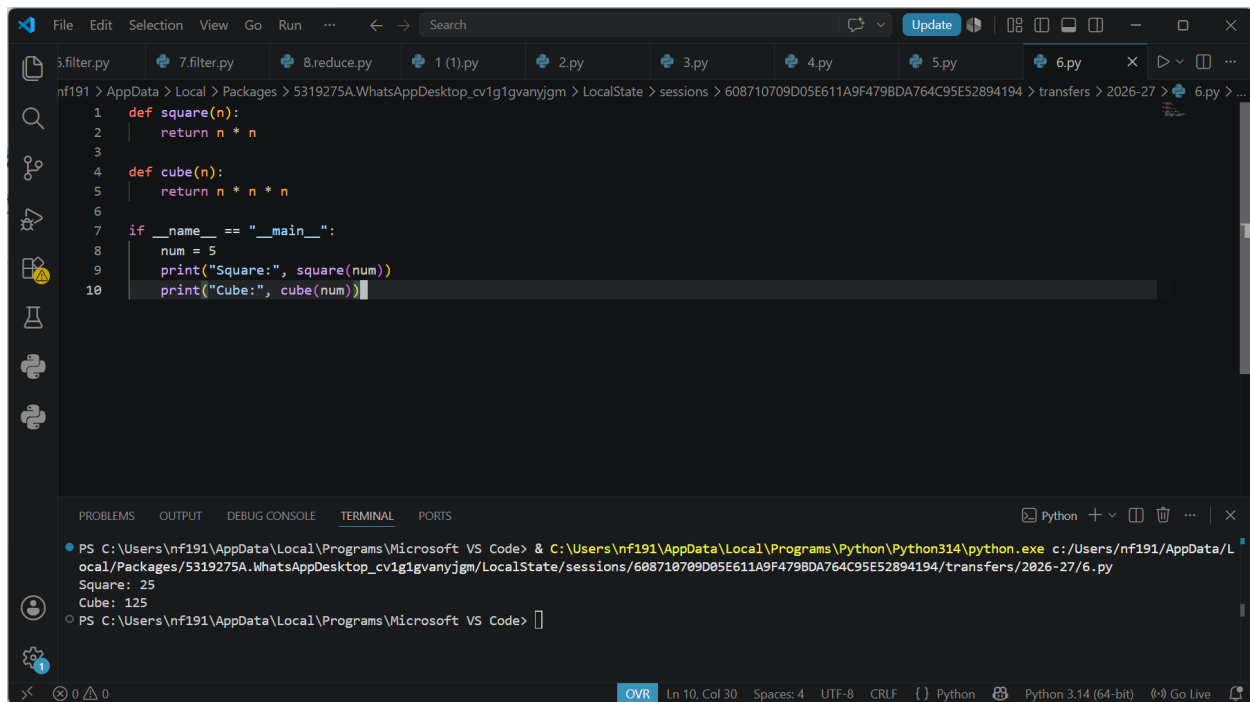
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nf191/AppData/Local/Packages/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F479BDA764C95E52894194/transfers/2026-27/5.py
Result: 25
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code>

```

Task 6:

Write a program to demonstrate the use of the `if __name__ == "__main__"` statement by calling functions to calculate and display the square and cube of a number when the file is executed directly.

Result:



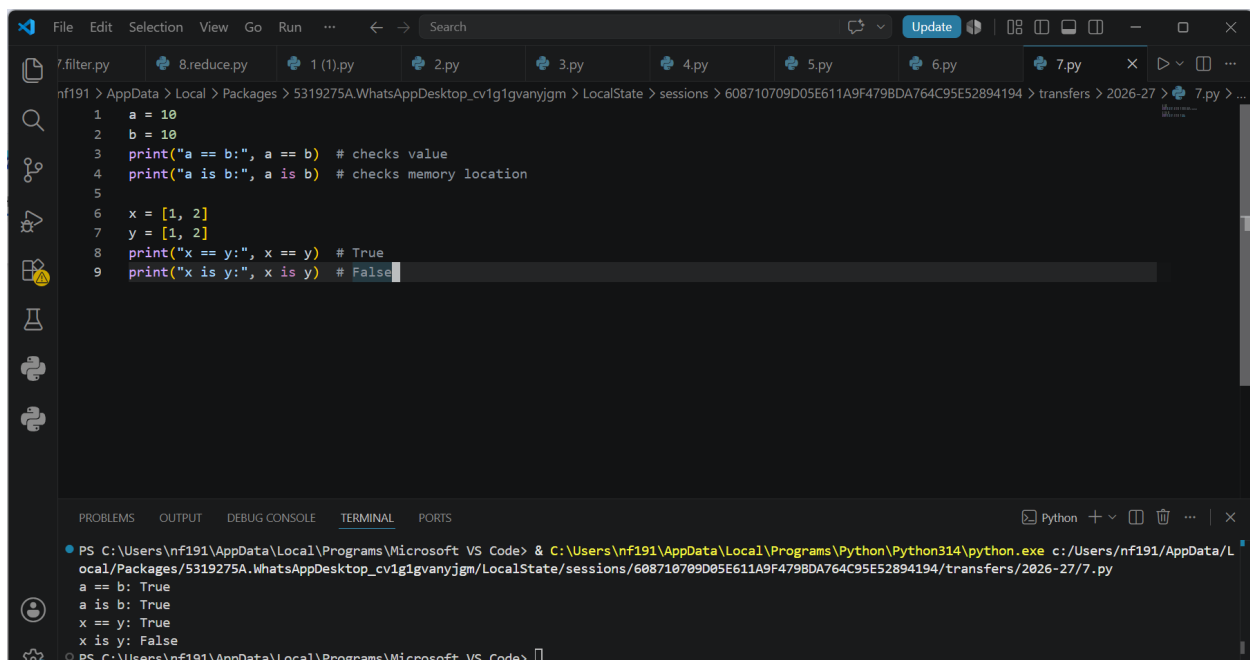
```
1 def square(n):
2     return n * n
3
4 def cube(n):
5     return n * n * n
6
7 if __name__ == "__main__":
8     num = 5
9     print("Square:", square(num))
10    print("Cube:", cube(num))
```

```
PS C:\Users\nf191\AppData\Local\Programs\Microsoft VS Code> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nf191/AppData/Local/Programs/5319275A.WhatsAppDesktop_cv1g1gvanyjgm/LocalState/sessions/608710709D05E611A9F479BDA764C95E52894194/transfers/2026-27/6.py
Square: 25
Cube: 125
```

Task 7:

Write a program to demonstrate the difference between the `==` operator and the `is` operator by comparing values and object identities of variables and lists.

Result:



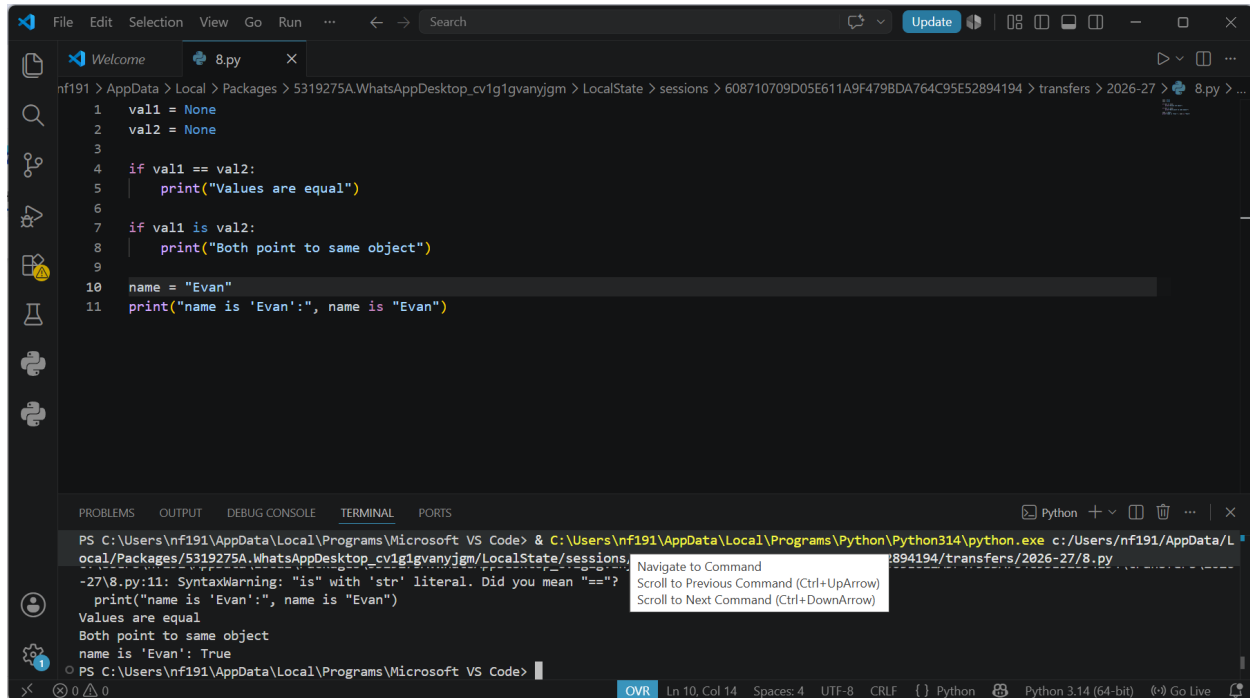
```
1 a = 10
2 b = 10
3 print("a == b:", a == b) # checks value
4 print("a is b:", a is b) # checks memory location
5
6 x = [1, 2]
7 y = [1, 2]
8 print("x == y:", x == y) # True
9 print("x is y:", x is y) # False
```

```
a == b: True
a is b: True
x == y: True
x is y: False
```

Task 8:

Write a program to demonstrate the use of the `is` and `==` operators by comparing `None` values and string objects, and observe the difference between value equality and object identity.

Result:



```
1 val1 = None
2 val2 = None
3
4 if val1 == val2:
5     print("Values are equal")
6
7 if val1 is val2:
8     print("Both point to same object")
9
10 name = "Evan"
11 print("name is 'Evan':", name is "Evan")
```

Output:

```
Values are equal
Both point to same object
name is 'Evan': True
```

The screenshot shows a VS Code editor with a Python file named `8.py`. The code defines two `None` variables, `val1` and `val2`, and compares them using `==` and `is` operators. It also defines a string variable `name` and compares it with a string literal using the `is` operator. The terminal output shows the results of these comparisons: "Values are equal", "Both point to same object", and "name is 'Evan': True". A tooltip is visible over the terminal, showing navigation options: "Navigate to Command", "Scroll to Previous Command (Ctrl+UpArrow)", and "Scroll to Next Command (Ctrl+DownArrow)".

Conclusion:

In this lab, we learned how to use the `os` module to perform basic file and directory operations. We understood the purpose of the `if __name__ == "__main__"` statement and how it controls program execution. We also explored the difference between the `==` and `is` operators by comparing object values and identities. This lab enhanced understanding of python modules, program structure, and object comparison techniques.